LAPORAN PRAKTIKUM STRUKTUR DATA

“Implementasi Abstract Data Type (ADT) Jam di Java”

disusun Oleh:

Az Zahrand Solichul Tajussalathin

NIM 2511532001

Dosen Pengampu: Dr. Wahyudi S.T M.T

Asisten Pratikum: Jovantri Immanuel Gulo

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

DAFTAR ISI

DAFTAR ISI i

KATA PENGANTAR ii

BAB I PENDAHULUAN 1

1.1 Latar Belakang 1

1.2 Tujuan Praktikum 1

1.3 Manfaat Praktikum 1

BAB II PEMBAHASAN 2

2.1 Program Jam 2

2.2 Program Driver 5

2.3 Program Mobil 6

2.4 Program MobilDriver 8

BAB III KESIMPULAN 10

3.1 Ringkasan Hasil Praktikum 10

DAFTAR PUSTAKA iii

KATA PENGANTAR

Puji syukur kehadirat Allah SWT atas limpahan rahmat dan karunia-Nya sehingga laporan praktikum dengan judul “Implementasi Abstract Data Type (ADT) Jam di Java” ini dapat disusun dan diselesaikan dengan baik. Laporan ini disusun sebagai salah satu bentuk pelaksanaan kegiatan praktikum pada mata kuliah Algoritma dan Pemrograman.

Ucapan terima kasih disampaikan kepada Bapak Dr. Wahyudi, S.T., M.T. selaku dosen pengampu, serta kepada asisten laboratorium yang telah memberikan bimbingan selama kegiatan praktikum berlangsung.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna, baik dari segi penulisan maupun isi. Oleh karena itu, saran dan kritik yang bersifat membangun sangat diharapkan demi perbaikan di masa yang akan datang. Harapannya, laporan ini dapat memberikan manfaat dan menjadi referensi bagi pembelajaran konsep perulangan dalam bahasa pemrograman Java.

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Dalam pemrograman berorientasi objek, konsep Abstract Data Type (ADT) merupakan salah satu fondasi penting yang memisahkan antara definisi suatu tipe data dengan implementasinya. ADT memungkinkan pemrogram untuk mendefinisikan sebuah tipe data beserta operasi-operasi yang dapat dilakukan terhadap data tersebut, tanpa perlu mengekspos detail implementasi internal kepada pengguna.

1.2 Tujuan Praktikum

Tujuan dari praktikum ini adalah:

1. Memahami konsep Abstract Data Type (ADT) dan penerapannya dalam bahasa pemrograman Java.
2. Mengimplementasikan ADT Jam dengan atribut jam, menit, dan detik beserta operasi-operasi yang relevan.
3. Menerapkan operasi aritmatika waktu seperti penjumlahan, pengurangan, dan navigasi detik menggunakan mekanisme konversi ke total detik.

1.3 Manfaat Praktikum

Manfaat yang diperoleh dari praktikum ini antara lain:

1. Menambah pemahaman mengenai prinsip Abstract Data Type dan hubungannya dengan konsep enkapsulasi dalam pemrograman berorientasi objek.
2. Melatih kemampuan dalam merancang kelas Java yang lengkap dengan validator, konstruktor, selektor, mutator, serta operasi aritmatika dan relasional.
3. Memberikan pengalaman dalam mengelola representasi waktu menggunakan pendekatan konversi detik untuk menyederhanakan operasi aritmatika.

BAB 2

PEMBAHASAN

2.1 Program Jam

```
package pekan1_2511532001;

public final class Jam_2511532001 {
	private int hh;
	private int mm;
	private int ss;

	//Validator
	public static boolean isValid(int h, int m, int s) {
		return (0 <= h && h <= 23) && (0 <= m && m <= 59) && (0 <= s
&& s <= 59);
	}

	//Konstruktor
	public Jam_2511532001(int h, int m, int s) {
		this.hh = h; this.mm = m; this.ss = s;
	}

	//Selektor
	public int getHour() { return hh; }
	public int getMinute() { return mm; }
	public int getSecond() { return ss; }

	//Mutator
	public void setHour(int h) {this.hh = h;}
	public void setMinute(int m) {this.mm = m;}
	public void setSecond(int s) {this.ss = s;}

	//Konversi
	public int toSeconds() { return hh * 3600 + mm * 60 + ss; }
	public static Jam_2511532001 fromSeconds(int total) {
	    if (total < 0) throw new IllegalArgumentException("detik
negatif");
	    total %= 24 * 3600; // wrap 24 jam
	    int h = total / 3600; total %= 3600;
	    int m = total / 60; int s = total % 60;
	    return new Jam_2511532001(h, m, s);
	}

	//Relasional
	public int compareTo(Jam_2511532001 other) { return
Integer.compare(this.toSeconds(), other.toSeconds()); }
	public boolean equals(Object o) {
	    if (!(o instanceof Jam_2511532001 j)) return false;
	    return hh == j.hh && mm == j.mm && ss == j.ss;
```

```
        }

        public int hashCode() { return java.util.Objects.hash(hh, mm, ss); }

        //Aritmatika
        public Jam_2511532001 plus(Jam_2511532001 other) { return
fromSeconds(this.toSeconds() + other.toSeconds()); }
        public Jam_2511532001 minus(Jam_2511532001 other) { return
fromSeconds(Math.floorMod(this.toSeconds() - other.toSeconds(), 24*3600));
}
        public Jam_2511532001 nextSecond() { return
fromSeconds(this.toSeconds() + 1); }
        public Jam_2511532001 nextNSeconds(int n) { return
fromSeconds(this.toSeconds() + Math.max(0, n)); }
        public Jam_2511532001 prevSecond() { return
fromSeconds(Math.floorMod(this.toSeconds() - 1, 24*3600)); }
        public Jam_2511532001 prevNSeconds(int n) { return
fromSeconds(Math.floorMod(this.toSeconds() - Math.max(0, n), 24*3600)); }

        //Durasi (detik) — bisa negatif jika this > other? sesuai
spesifikasi durasi (JAw, JAKh)
        public static int durasiDetik(Jam_2511532001 jaw, Jam_2511532001
jakh) { return jakh.toSeconds() - jaw.toSeconds(); }

        // overriding ke string
        public String toString() { return String.format("%02d:%02d:%02d",
hh, mm, ss); }

        public class JamDriver {
                public static void main (String[]args) {
                        Jam_2511532001 a = new Jam_2511532001(23, 59, 50);
                        Jam_2511532001 b = new Jam_2511532001(0, 0, 15);
                        System.out.println("a = " + a);
                        System.out.println("b = " + b);
                        System.out.println("a+b = " + a.plus (b));
                        System.out.println("next 20s = " + a.nextNSeconds(20));
                        System.out.println("durasi(a,b) = " +
Jam_2511532001.durasiDetik(a, b));
                        System.out.println("a.compareTo(b) = " +
a.compareTo(b));
                }
        }
}
```

Program di atas merupakan implementasi ADT Jam yang direpresentasikan sebagai kelas final dalam bahasa pemrograman Java. Penggunaan keyword final pada kelas ini bertujuan untuk mencegah pewarisan (inheritance), sehingga menjamin bahwa perilaku dan kontrak ADT tidak dapat diubah oleh subclass manapun. Kelas ini memiliki tiga atribut private yaitu hh (jam), mm (menit), dan ss (detik), yang merepresentasikan waktu dalam format 24 jam.

Validator merupakan komponen pertama yang dirancang dalam ADT ini. Method isValid(int h, int m, int s) bersifat static dan digunakan untuk memvalidasi apakah nilai jam (0–23), menit (0–59), dan detik (0–59) berada dalam rentang yang benar sebelum sebuah objek Jam dibuat.

Konstruktor Jam (int h, int m, int s) menerima tiga parameter berupa nilai jam, menit, dan detik, kemudian menyimpannya ke dalam atribut instance. Adapun selektor (getHour, getMinute, getSecond) dan mutator (setHour, setMinute, setSecond) masing-masing berfungsi untuk mengakses dan mengubah nilai atribut secara terpisah sesuai prinsip enkapsulasi.

Operasi Konversi menjadi fondasi dari seluruh operasi aritmatika pada ADT ini. Method toSeconds() mengonversi waktu ke dalam total detik. Sebaliknya, method static fromSeconds(int total) mengonversi total detik kembali menjadi objek Jam dengan melakukan operasi modulo 24 jam terlebih dahulu untuk memastikan nilai hasil selalu berada dalam satu siklus hari.

Operasi Relasional diimplementasikan melalui tiga method. Method compareTo membandingkan dua objek Jam berdasarkan total detiknya menggunakan Integer.compare. Method equals melakukan perbandingan kesetaraan dengan memeriksa kesamaan nilai hh, mm, dan ss secara langsung. Method hashCode menggunakan Objects.hash untuk menghasilkan hash yang konsisten dengan kontrak equals di Java.

Operasi Aritmatika memanfaatkan konversi detik. Method plus menjumlahkan total detik kedua objek lalu mengonversinya kembali, sementara method minus, prevSecond, dan prevNSeconds menggunakan Math.floorMod untuk menangani selisih negatif agar hasil selalu berada dalam rentang 0 hingga 86399 detik. Method durasiDetik menghitung selisih detik dan dapat bernilai negatif apabila JAw lebih besar dari JAKh.

Method toString melakukan override terhadap method bawaan Object untuk menghasilkan representasi string waktu dalam format HH:MM:SS

2.2 Program Driver

```
package tugas;

public class Driver_2511532001 {
 public static void main(String[] args) {
     Mobil_2511532001 daftar = new Mobil_2511532001();

     Mobil_2511532001 m1 = new Mobil_2511532001("Avanza", 2020, 1300,
150000000, "Toyota");
     Mobil_2511532001 m2 = new Mobil_2511532001("Brio", 2021, 1200,
180000000, "Honda");
     Mobil_2511532001 m3 = new Mobil_2511532001("Xpander", 2022, 1500,
250000000, "Mitsubishi");

     daftar.tambahMobil(m1);
     daftar.tambahMobil(m2);
     daftar.tambahMobil(m3);

     daftar.tampilkanSemua();

     System.out.println("\nNama mobil pertama: " + m1.getNama());
     System.out.println("Harga: Rp" + (long) m1.getHarga());

     m1.setHarga(155000000);
     System.out.println("Harga setelah update: Rp" + (long) m1.getHarga());

     daftar.hapusMobil("Brio");

     daftar.tampilkanSemua();
 }
}
```

JamDriver2 merupakan program driver interaktif yang memungkinkan pengguna memasukkan dua buah objek Jam secara langsung melalui terminal menggunakan java.util.Scanner

Komponen utama program ini adalah method pembantu buatJamDariInput(Scanner sc) yang mengimplementasikan pola *input loop with validation*. Method ini terus meminta input kepada pengguna hingga nilai yang dimasukkan lolos validasi melalui pemanggilan Jam_2511532001.isValid(h, m, s). Jika input tidak valid, program mencetak pesan error dan memulai ulang proses input tanpa menghentikan program.

Pada bagian utama main, program secara berurutan menjalankan enam kelompok operasi: (1) input Jam 1, (2) input Jam 2, (3) tampilan representasi string dan total

detik, (4) perbandingan relasional menggunakan compareTo dengan interpretasi hasilnya dalam bahasa natural, (5) operasi durasi, nextSecond, dan prevSecond, serta (6) penjumlahan Jam 1 dan Jam 2. Setelah semua operasi selesai, input.close() dipanggil untuk menutup stream Scanner.

2.3 Program Mobil

```
package pekan1_2511532001;

import java.util.ArrayList;

public class Mobil_2511532001 {

        private String nama;
        private int tahun;
        private int cc;
        private double harga;
        private String merk;

        private ArrayList<Mobil_2511532001> daftarMobil;

        public Mobil_2511532001() {
            this.daftarMobil = new ArrayList<>();
        }

        public Mobil_2511532001(String nama, int tahun, int cc, double
harga, String merk) {
            this.nama = nama;
            this.tahun = tahun;
            this.cc = cc;
            this.harga = harga;
            this.merk = merk;
        }

        public String getNama() { return nama; }
        public int getTahun() { return tahun; }
        public int getCc() { return cc; }
        public double getHarga() { return harga; }
        public String getMerk() { return merk; }
        public ArrayList<Mobil_2511532001> getDaftarMobil() { return
daftarMobil; }

        public void setNama(String nama) { this.nama = nama; }
        public void setTahun(int tahun) { this.tahun = tahun; }
        public void setCc(int cc) { this.cc = cc; }
        public void setHarga(double harga) { this.harga = harga; }
        public void setMerk(String merk) { this.merk = merk; }

        public void tambahMobil(Mobil_2511532001 m) {
```

```
            daftarMobil.add(m);
            System.out.println("Mobil " + m.getNama() + " berhasil
ditambahkan.");
        }

        public void hapusMobil(String namaMobil) {
            boolean ditemukan = false;
            for (int i = 0; i < daftarMobil.size(); i++) {
                if
(daftarMobil.get(i).getNama().equalsIgnoreCase(namaMobil)) {
                    daftarMobil.remove(i);
                    System.out.println("Mobil " + namaMobil + " berhasil
dihapus.");
                    ditemukan = true;
                    break;
                }
            }
            if (!ditemukan) {
                System.out.println("Mobil " + namaMobil + " tidak
ditemukan.");
            }
        }

        public void tampilkanSemua() {
            System.out.println("\n===== Daftar Mobil =====");
            if (daftarMobil.isEmpty()) {
                System.out.println("Daftar kosong.");
            } else {
                for (Mobil_2511532001 m : daftarMobil) {
                    System.out.println(m.toString());
                }
            }
        }

        public String toString() {
            return String.format("Nama: %s | Tahun: %d | CC: %d | Harga:
Rp%.0f | Merk: %s",
                    nama, tahun, cc, harga, merk);
        }
       }
```

Class Mobil digunakan untuk merepresentasikan data mobil sekaligus menyimpan kumpulan objek mobil dalam sebuah ArrayList. Atribut yang dimiliki meliputi nama, tahun, cc, harga, dan merk. Selain itu, class ini juga memiliki atribut daftarMobil yang bertipe ArrayList<Mobil> untuk menampung banyak objek mobil.

Terdapat dua konstruktor pada class ini. Konstruktor tanpa parameter digunakan untuk membuat objek daftar yang berfungsi sebagai wadah penyimpanan

data mobil. Sementara itu, konstruktor berparameter digunakan untuk membuat objek mobil dengan data tertentu.

Method getter dan setter disediakan untuk membaca serta mengubah atribut mobil. Method tambahMobil() digunakan untuk menambahkan objek mobil ke dalam ArrayList, sedangkan method hapusMobil() digunakan untuk menghapus mobil berdasarkan nama. Method tampilkanSemua() berfungsi menampilkan seluruh data mobil yang tersimpan di dalam daftar. Method toString() di-override agar data mobil dapat ditampilkan dalam format yang lebih rapi dan mudah dibaca.

2.4 Program MobilDriver

```
package pekan1_2511532001;

public class MobilDriver_2511532001 {
 public static void main(String[] args) {
     Mobil_2511532001 daftar = new Mobil_2511532001();

     Mobil_2511532001 m1 = new Mobil_2511532001("Avanza", 2020, 1300,
150000000, "Toyota");
     Mobil_2511532001 m2 = new Mobil_2511532001("Brio", 2021, 1200,
180000000, "Honda");
     Mobil_2511532001 m3 = new Mobil_2511532001("Xpander", 2022, 1500,
250000000, "Mitsubishi");

     daftar.tambahMobil(m1);
     daftar.tambahMobil(m2);
     daftar.tambahMobil(m3);

     daftar.tampilkanSemua();

     System.out.println("\nNama mobil pertama: " + m1.getNama());
     System.out.println("Harga: Rp" + (long) m1.getHarga());

     m1.setHarga(155000000);
     System.out.println("Harga setelah update: Rp" + (long) m1.getHarga());

     daftar.hapusMobil("Brio");

     daftar.tampilkanSemua();
 }
}
```

Program MobilDriver berfungsi untuk menguji penggunaan class Mobil. Pada program ini, dibuat sebuah objek daftar bernama daftar yang akan menampung data

mobil. Selanjutnya dibuat tiga objek mobil, yaitu Avanza, Brio, dan Xpander, dengan atribut yang berbeda-beda.

Ketiga objek tersebut kemudian ditambahkan ke dalam daftar menggunakan method tambahMobil(). Setelah itu, program menampilkan seluruh daftar mobil. Program juga mendemonstrasikan penggunaan getter untuk menampilkan nama dan harga mobil pertama, serta setter untuk mengubah harga mobil tersebut.

Selanjutnya, program menghapus data mobil dengan nama “Brio” menggunakan method hapusMobil(). Setelah penghapusan dilakukan, daftar mobil ditampilkan kembali sehingga dapat terlihat bahwa data Brio sudah tidak ada lagi.

BAB 3

KESIMPULAN

3.1 Ringkasan Hasil Praktikum

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa implementasi Abstract Data Type (ADT) Jam menggunakan bahasa pemrograman Java memberikan pemahaman mendalam mengenai konsep enkapsulasi dan pemisahan antara spesifikasi tipe data dengan implementasinya, diketahui bahwa sebuah ADT yang baik terdiri atas validator, konstruktor, selektor, mutator, operasi konversi, operasi relasional, dan operasi aritmatika yang saling terintegrasi.

Penggunaan strategi konversi total detik sebagai jembatan aritmatika terbukti efektif dalam menyederhanakan operasi penjumlahan dan pengurangan waktu yang melibatkan perpindahan lintas komponen (detik ke menit, menit ke jam). Penerapan Math.floorMod menjadi kunci dalam menangani skenario edge case seperti pengurangan waktu yang menghasilkan nilai negatif atau melewati batas tengah malam (midnight wrap-around).

DAFTAR PUSTAKA

[1] W3Schools, “Java Data Structures”. [Daring].

Tersedia pada: https://www.w3schools.com/java/java_data_structures.asp.

[Diakses: 1 April 2026].